HTTP access control (CORS)

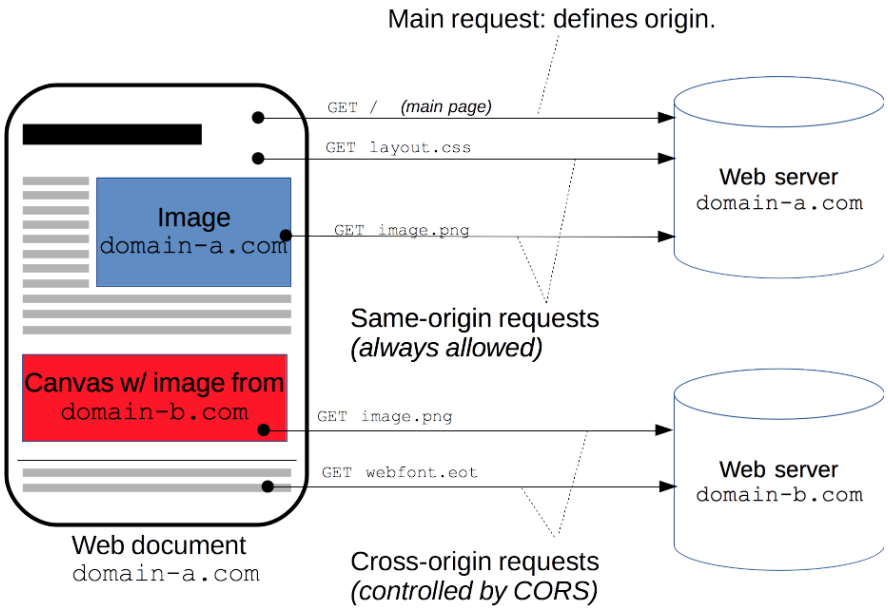
Cross-Origin Resource Sharing (CORS) is like a security rule for web browsers. It controls when a website from one place can ask another website for information.

Here's how it plays out:

1. **Asking Permission:** When one website wants to grab something (like an image or data) from another website, the second website gets to decide if that's okay. CORS helps with this decision.
2. **Testing the Waters:** Before actually fetching the data, the browser sends a little message to the second website, sort of like saying, "Hey, can I get this data from you?" This is the "preflight" request.
3. **Laying Out the Plan:** The browser also explains to the second website how it's planning to get the data. It's like saying, "I'm going to ask nicely using these methods and rules."
4. **Getting Permission:** If the second website agrees, it sends a special signal back to the browser (these are the CORS headers) saying, "Sure, you can go ahead and fetch that data."
5. **Safety First:** If the special signal isn't there, browsers are cautious and won't allow websites to grab data from other places without permission. This adds a layer of security to ensure that websites can't just take data from anywhere they want.

Picture this: If one website wants to show an image from another site, CORS ensures there's a clear understanding between the sites and the browser that everything is safe and authorized. This way, we prevent unauthorized data access and maintain a safer internet environment.

For security reasons, browsers restrict cross-origin HTTP requests initiated from scripts. For example, `XMLHttpRequest` and the Fetch API follow the same-origin policy. This means that a web application using those APIs can only request resources from the same origin the application was loaded from unless the response from other origins includes the right CORS headers.



The CORS mechanism supports secure cross-origin requests and data transfers between browsers and servers. Modern browsers use CORS in APIs such as `XMLHttpRequest` or Fetch to mitigate the risks of cross-origin HTTP requests.

What requests use CORS?

This [cross-origin sharing standard] can enable cross-origin HTTP requests for:

- Invocations of the `XMLHttpRequest` or Fetch APIs, as discussed above.
- Web Fonts (for cross-domain font usage in @font-face within CSS), so that servers can deploy TrueType fonts that can only be loaded cross-origin and used by websites that are permitted to do so.
- WebGL textures.
- Images/video frames drawn to a canvas using `drawImage()`.
- CSS Shapes from images.

This is a general article about Cross-Origin Resource Sharing and includes a discussion of the necessary HTTP headers.

Functional overview

The Cross-Origin Resource Sharing standard works by adding new HTTP headers that let servers describe which origins are permitted to read that information from a web browser. Additionally, for HTTP request methods that can cause side-effects on server data (in particular, HTTP methods other than `GET`, or `POST` with certain MIME types), the specification mandates that browsers "preflight" the request, soliciting supported methods from the server with the HTTP `OPTIONS` request method, and then, upon "approval" from the server, sending the actual request. Servers can also inform clients whether "credentials" (such as Cookies and HTTP Authentication) should be sent with requests.

CORS failures result in errors but for security reasons, specifics about the error *are not available to JavaScript*. All the code knows is that an error occurred. The only way to determine what specifically went wrong is to look at the browser's console for details.

Subsequent sections discuss scenarios, as well as provide a breakdown of the HTTP headers used.

Examples of access control scenarios

We present three scenarios that demonstrate how Cross-Origin Resource Sharing works. All these examples use `XMLHttpRequest`, which can make cross-origin requests in any supporting browser.

Simple Requests

Some requests don't need a special permission check called CORS preflight. These are called "simple requests." They are straightforward and don't need extra security checks before being sent.

In the past, web forms could send data to any website. This means servers are already prepared to handle security issues like cross-site request forgery (CSRF). So, if a request looks like a form submission, servers don't need to double-check before accepting it. However, the server still needs to allow the response to be shared with the requesting script.

Conditions for Simple Requests:

- Allowed methods: GET, HEAD, POST
- Only certain headers can be added manually (Accept, Accept-Language, Content-Language, Content-Type, Range).
- The Content-Type header must use specific types (application/x-www-form-urlencoded, multipart/form-data, text/plain).
- Certain technical conditions must be met, like not using certain types of objects or event listeners.

Example: For example, suppose web content at https://foo.example wishes to invoke content on domain https://bar.other. Code of this sort might be used in JavaScript deployed on foo.example. This is a simple request scenario:

```
const xhr = new XMLHttpRequest();
const url = "https://bar.other/resources/public-data/";

xhr.open("GET", url);
xhr.onreadystatechange = someHandler;
xhr.send();
```

Let's look at what the browser will send to the server in this case:

```
GET /resources/public-data/ HTTP/1.1
Host: bar.other
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.14; rv:71.0) Gecko/20100101
            Firefox/71.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Connection: keep-alive
Origin: https://foo.example
```

The request header of note is Origin, which shows that the invocation is coming from `https://foo.example`.

Now let's see how the server responds:

```
HTTP/1.1 200 OK
Date: Mon, 01 Dec 2008 00:23:53 GMT
Server: Apache/2
Access-Control-Allow-Origin: *
Keep-Alive: timeout=2, max=100
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: application/xml

[…XML Data…]
```

The browser sends an "Origin" header indicating the source (`https://foo.example`), and the server responds with an "Access-Control-Allow-Origin" header, which specifies who can access the resource. If the server allows all origins, it uses "*", but it can also specify a specific origin for more control.

Summary: Simple requests are basic requests that don't need extra security checks. They're used for getting or sending data between different websites. Servers already handle security for these requests, so they don't need extra verification, except for specifying who can access the response.
Remember: If a more secure or complex request is needed, additional steps might be required, and servers can set specific rules for which origins are allowed to access their resources.

Preflighted requests

Preflighted requests are a special type of request that browsers send to a different website before making the actual request. This is done to ensure that the actual request is safe and won't cause any issues with user data.

Preflight happens when you're trying to do something a bit more complex, like sending special headers or using certain methods in your request. Browsers check with the server first to see if everything is okay.

Example Scenario: Let's say you want to send data to a different website using JavaScript. If you're doing a simple request, like just getting data using the `GET` method, there's no preflight needed. But if you're sending more specific headers or using methods like `POST`, the browser might need to do a preflight.

How Preflight Works:

1. Browser sends a special `OPTIONS` request to the server before the real request.
2. This `OPTIONS` request asks the server if it's okay to send the real request with the given headers and method.
3. The server responds, saying what's allowed and what's not.
4. If the server approves, the browser sends the actual request.

Imagine you want to send some XML data to another site using JavaScript. The browser might first ask the server if it's okay to do this. If the server says yes, the browser will send the XML data.

Preflight Request example:

```
OPTIONS /api/data HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:89.0) Gecko/20100101
            Firefox/89.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Access-Control-Request-Method: POST
Access-Control-Request-Headers: X-Custom-Header
Origin: https://your-site.com
Connection: keep-alive
```

Preflight Response example:

```
HTTP/1.1 200 OK
Date: Mon, 01 Jan 2023 12:34:56 GMT
Server: Apache/2.4.39 (Unix)
Access-Control-Allow-Origin: https://your-site.com
Access-Control-Allow-Methods: POST, GET, OPTIONS
Access-Control-Allow-Headers: X-Custom-Header
Access-Control-Max-Age: 86400
Content-Length: 0
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/plain; charset=UTF-8
```

In the example above:

- The browser sends an OPTIONS request (preflight request) to `example.com` to ask if it's okay to send a `POST` request with the custom header `X-Custom-Header`.
- The server responds with a 200 OK status and includes the necessary CORS headers (`Access-Control-Allow-Origin`, `Access-Control-Allow-Methods`, `Access-Control-Allow-Headers`, and `Access-Control-Max-Age`) to indicate that the preflighted request is allowed.

This demonstrates how the preflight process works by checking with the server whether a more complex request is permitted.

Why is Preflight Important? Preflight helps keep things safe and secure. It ensures that any special requests you make won't accidentally mess up user data or cause other problems.

CORS and Redirects: Sometimes, if a preflight request is followed by a redirect, not all browsers handle it the same way. Some might show an error. To work around this, you might need to change the way your requests are structured or the way the server responds.

Remember: As a front-end developer, you don't need to be a preflight expert. Just know that browsers might send these special preflight requests when you're doing more advanced stuff with requests, and understand the basic idea of how they help ensure safe communication between websites.

Requests with Credentials

Credentialed requests are special requests that know about cookies and authentication info. They're used when you need to send sensitive information securely between different websites.

By default, browsers don't send cookies and authentication info in requests to other websites. But you can tell the browser to include them with a special flag.

Let's say you want to get some data from a different website along with your authentication details. You'll use credentialed requests to make sure your info is sent securely.

Using Credentialed Requests: In JavaScript, when you're making a request using XMLHttpRequest or Fetch, you can set the `withCredentials` flag to `true` to include cookies and authentication info.

Code Example - Using `XMLHttpRequest`:

```
const xhr = new XMLHttpRequest();
const url = "https://bar.other/resources/credentialed-content/";

function callOtherDomain() {
  if (xhr) {
    xhr.open("GET", url, true);
    xhr.withCredentials = true; // Set to true to include credentials
    xhr.onreadystatechange = handler;
    xhr.send();
  }
}
```

The server you're requesting data from needs to be set up to allow credentialed requests. It should respond with the `Access-Control-Allow-Credentials: true` header.

Credentialed requests are used to send sensitive data between websites securely. You set a flag in your request code to include cookies and authentication info. This ensures that your data is treated with care and doesn't end up in the wrong hands. Keep in mind that third-party cookie policies still apply, so there are some limitations to consider.

The HTTP response headers

This section lists the HTTP response headers that servers return for access control requests as defined by the Cross-Origin Resource Sharing specification. The previous section gives an overview of these in action.

Access-Control-Allow-Origin

A returned resource may have one `Access-Control-Allow-Origin` header with the following syntax:

```
Access-Control-Allow-Origin: <origin> | *
```

`Access-Control-Allow-Origin` specifies either a single origin which tells browsers to allow that origin to access the resource, or else — for requests **without** credentials — the " * " wildcard tells browsers to allow any origin to access the resource.

For example, to allow code from the origin `https://mozilla.org` to access the resource, you can specify:

```
Access-Control-Allow-Origin: https://mozilla.org
Vary: Origin
```

If the server specifies a single origin (that may dynamically change based on the requesting origin as part of an allowlist) rather than the " * " wildcard, then the server should also include `Origin` in the `Vary` response header to indicate to clients that server responses will differ based on the value of the `Origin` request header.

Access-Control-Expose-Headers

The `Access-Control-Expose-Headers` header adds the specified headers to the allowlist that JavaScript (such as `getResponseHeader()`) in browsers is allowed to access.

```
Access-Control-Expose-Headers: <header-name>[, <header-name>]*
```

For example, the following:

```
Access-Control-Expose-Headers: X-My-Custom-Header, X-Another-Custom-Header
```

...would allow the X-My-Custom-Header and X-Another-Custom-Header headers to be exposed to the browser.

Access-Control-Max-Age

The `Access-Control-Max-Age` header indicates how long the results of a preflight request can be cached. For an example of a preflight request, see the above examples.

```
Access-Control-Max-Age: <delta-seconds>
```

The delta-seconds parameter indicates the number of seconds the results can be cached.

Access-Control-Allow-Credentials

The `Access-Control-Allow-Credentials` header indicates whether or not the response to the request can be exposed when the `credentials` flag is true. When used as part of a response to a preflight request, this indicates whether or not the actual request can be made using credentials. Note that simple `GET` requests are not preflighted, and so if a request is made for a resource with credentials, if this header is not returned with the resource, the response is ignored by the browser and not returned to web content.

```
Access-Control-Allow-Credentials: true
```

Credentialed requests are discussed above.

Access-Control-Allow-Methods

The `Access-Control-Allow-Methods` header specifies the method or methods allowed when accessing the resource. This is used in response to a preflight request. The conditions under which a request is preflighted are discussed above.

```
Access-Control-Allow-Methods: <method>[, <method>]*
```

An example of a preflight request is given above, including an example which sends this header to the browser.

Access-Control-Allow-Headers

The `Access-Control-Allow-Headers` header is used in response to a preflight request to indicate which HTTP headers can be used when making the actual request. This header is the server side response to the browser's `Access-Control-Request-Headers` header.

```
Access-Control-Allow-Headers: <header-name>[, <header-name>]*
```

The HTTP request headers

This section lists headers that clients may use when issuing HTTP requests in order to make use of the cross-origin sharing feature. Note that these headers are set for you when making invocations to servers. Developers using cross-origin `XMLHttpRequest` capability do not have to set any cross-origin sharing request headers programmatically.

Origin

The `Origin` header indicates the origin of the cross-origin access request or preflight request.

```
Origin: <origin>
```

The origin is a URL indicating the server from which the request is initiated. It does not include any path information, only the server name.

Note: The `origin` value can be `null`.

Note that in any access control request, the `Origin` header is **always** sent.

Access-Control-Request-Method

The `Access-Control-Request-Method` is used when issuing a preflight request to let the server know what HTTP method will be used when the actual request is made.

```
Access-Control-Request-Method: <method>
```

Examples of this usage can be found above in the preflighted requests lesson.

Access-Control-Request-Headers

The `Access-Control-Request-Headers` header is used when issuing a preflight request to let the server know what HTTP headers will be used when the actual request is made (such as with `setRequestHeader()`). This browser-side header will be answered by the complementary server-side header of `Access-Control-Allow-Headers`.

```
Access-Control-Request-Headers: <field-name>[, <field-name>]*
```

Examples of this usage can be found above in the preflighted requests lesson.

This lesson was adapted from MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS